

LUMEX

Academic Research Publishing Platform

A full-stack scholarly publishing system supporting article submission, peer review, editorial workflows, payments, AI-powered tools, and open access publishing.

Version	1.0.0
Author	Rahul Mishra
Date	June 2026
Stack	React + Express + Prisma + PostgreSQL
Database	Neon PostgreSQL (Cloud)
File Storage	Supabase Storage
AI Engine	Groq (Llama 3.3 70B)

Comprehensive Project Report

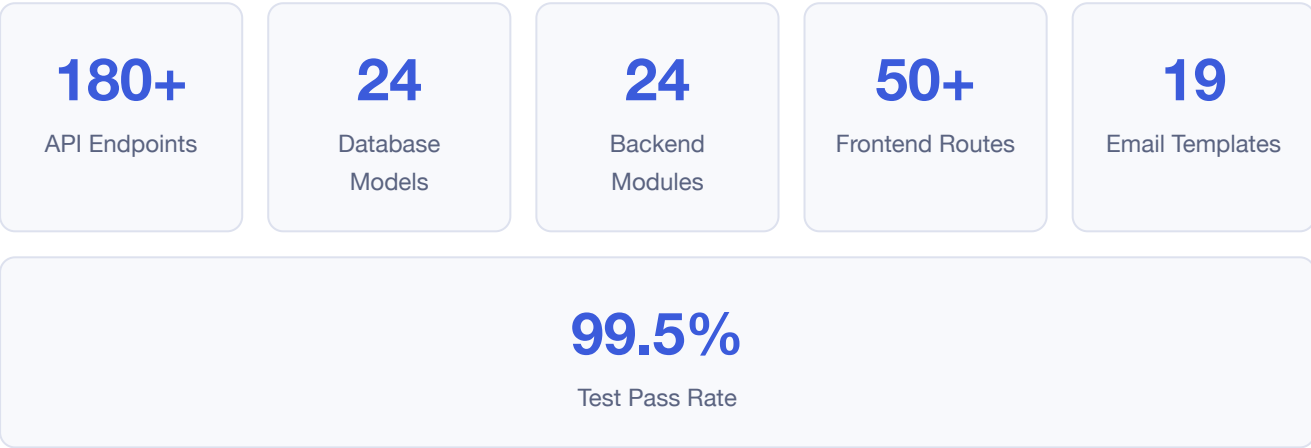
Table of Contents

1. [Executive Overview](#)
2. [System Architecture](#)
3. [Technology Stack](#)
4. [Backend Architecture](#)
 1. Module Structure
 2. API Endpoints (180+)
 3. Middleware Pipeline
 4. Authentication & Authorization
 5. Database Schema (24 Models)
 6. Workflow State Machines
 7. Email & Notification System
 8. Caching & Rate Limiting
 9. File Storage (Supabase)
 10. AI Module (Groq)
5. [Frontend Architecture](#)
 1. Feature-Sliced Design
 2. Routing (50+ Routes)
 3. State Management (Zustand + TanStack Query)
 4. API Layer & Fallback Pattern
 5. Component Hierarchy
 6. Design System & Theming
 7. Role-Based Access Control
6. [Database Design](#)
7. [Frontend-Backend Integration Status](#)
8. [Testing & Quality](#)
9. [Deployment & Environment](#)

10. Roadmap & Next Steps

1. Executive Overview

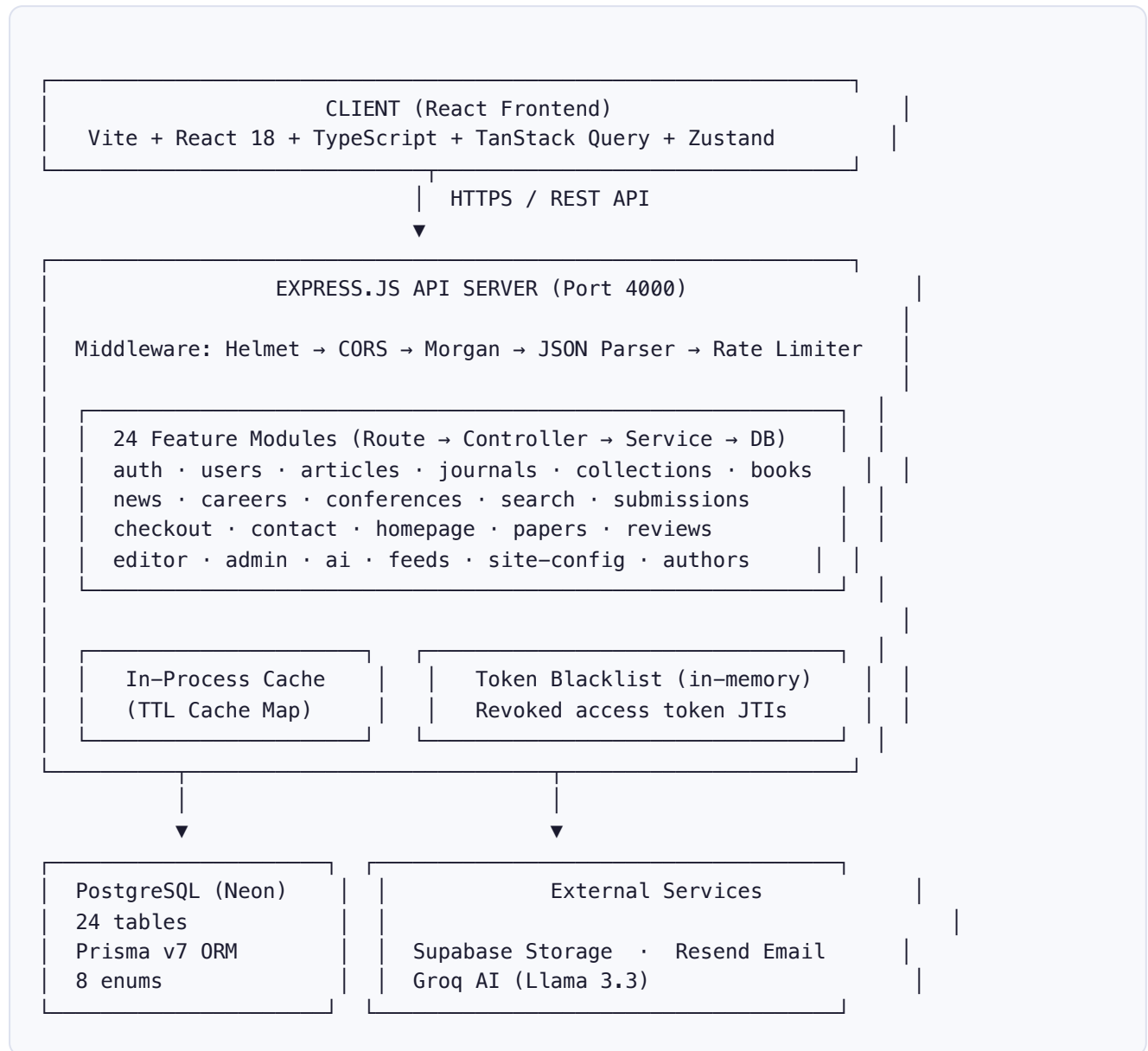
Lumex is a full-featured academic research publishing platform designed to handle the complete lifecycle of scholarly communication — from manuscript submission and peer review through editorial decision-making, publication, and reader access. The platform supports multiple user roles (Reader, Author, Reviewer, Editor, Admin), implements dual publishing workflows (legacy paper system + modern submission pipeline), and integrates AI-powered paper summarization.



Key Capabilities

- **Multi-Role System:** 5 roles (READER, AUTHOR, REVIEWER, EDITOR, ADMIN) with granular permissions
- **Dual Submission Workflows:** Legacy paper system + modern 8-state submission pipeline
- **Peer Review:** Editor assigns reviewers, structured review form, editorial decision system
- **Publishing:** Journals, books, chapters, collections, special issues, conferences
- **AI Integration:** PDF summarization and Q&A using Groq (Llama 3.3 70B)
- **Payment System:** Article purchase and APC (Article Processing Charges) checkout
- **Search:** Full-text search with faceted filtering and suggestions
- **Email Notifications:** 19 automated email templates via Resend
- **Alert System:** Keyword/journal/discipline alerts with batch digest delivery

2. System Architecture



3. Technology Stack

3.1 Backend Stack

Layer	Technology	Version	Purpose
Runtime	Node.js (ESM)	20+	Server runtime with ES module support
Language	TypeScript	5.9.3	Type-safe development, strict mode
Framework	Express	5.2.1	HTTP server, async error propagation
ORM	Prisma	7.4+	Type-safe database access, migrations
Database	PostgreSQL (Neon)	14+	Cloud-hosted relational database
DB Adapter	@prisma/adapters-neon / pg	7.x	Runtime adapter selection (Neon or local)
Auth	jsonwebtoken + bcrypt	9.0 / 6.0	JWT access/refresh tokens, password hashing
Validation	Zod	4.3.6	Request body/query schema validation
File Upload	multer	2.1.1	Multipart form-data, memory storage
File Storage	Supabase Storage	2.98+	PDF/DOCX file hosting with CDN
Email	Resend	6.9.3	Transactional emails, 19 templates
AI	Groq SDK	1.2.1	Paper summarization & Q&A (Llama 3.3 70B)
PDF Parsing	pdf-parse	1.1.1	Extract text from uploaded PDFs
Security	Helmet + CORS + rate-limit	8.x / 2.x / 8.x	HTTP headers, CORS, request throttling
Logging	Morgan	1.10.1	HTTP request logging

3.2 Frontend Stack

Layer	Technology	Version	Purpose
Build Tool	Vite	7.3.1	Fast HMR, code splitting, bundling
UI Library	React	18.3.0	Component-based UI with hooks
Language	TypeScript	5.9.3	Strict type safety

Layer	Technology	Version	Purpose
Routing	React Router	7.13.1	Data router with lazy loading
Server State	TanStack React Query	5.90.21	API data fetching, caching, sync
Client State	Zustand	5.0.11	Lightweight global state management
Styling	Tailwind CSS	3.4.0	Utility-first CSS framework
UI Primitives	Radix UI	Various	10 accessible headless components
Animation	Framer Motion	12.34.3	Page transitions, micro-interactions
Forms	React Hook Form + Zod	7.71 / 4.3	Form state management + validation
PDF Viewer	react-pdf + pdfjs-dist	10.4 / 5.4	In-browser PDF rendering
Math	KaTeX	0.16.33	LaTeX math notation rendering
Markdown	react-markdown + remark-gfm	10.1 / 4.0	Rich text rendering
SEO	React Helmet Async	2.0.5	Document head management
Testing	Vitest + Playwright	4.0 / 1.58	Unit + E2E testing
Code Quality	ESLint + Prettier + Husky	9.x / 3.8 / 9.1	Linting, formatting, pre-commit hooks

4. Backend Architecture

4.1 Module Structure

Every backend module follows a strict 4-layer pattern:

```
modules/<feature>/
├── <feature>.routes.ts      Route definitions + middleware chaining
├── <feature>.controller.ts  Parse request → call service → sendSuccess()
├── <feature>.service.ts     Business logic + Prisma + cache + email
└── <feature>.schema.ts     Zod schemas for validation
```

4.2 Backend Modules & API Endpoints

Module	Base Path	Key Endpoints	Auth Required
auth	/api/auth	register, login, logout, refresh, forgot-password, reset-password, verify-email, me	Partial
users	/api/users	me, dashboard, submissions, orders, alerts, saved-articles, reading-history	Yes
articles	/api/articles	list, top, :doi detail, metrics, cite, download, publish (admin)	Partial
journals	/api/journals	list, :slug detail, articles, issues, editorial-board, special-issues, admin CRUD	Partial
collections	/api/collections	list, :slug detail, articles, admin CRUD	Partial
books	/api/books	list, :isbn detail, chapters, admin CRUD	Partial
news	/api/news	list, :slug detail, admin CRUD	Partial
careers	/api/careers	list, :slug detail, admin CRUD	Partial
conferences	/api/conferences	list, :slug detail, admin CRUD	Partial
search	/api/search	full-text search, suggestions, related	No
submissions	/api/submissions	create, edit, submit, files, under-review, decision, withdraw, revise	Yes
checkout	/api/checkout	article purchase, APC payment, orders, verify	Yes
contact	/api/contact	submit form, admin list, status update	Partial
homepage	/api/homepage	aggregated homepage data (cached)	No

Module	Base Path	Key Endpoints	Auth Required
site-config	/api/site-config	get/set config key-value pairs	Partial
papers	/papers	Legacy: CRUD, submit, upload, approve/reject	Yes
reviews	/reviews	Legacy: reviewer dashboard, assignments, submit review	Yes
editor	/editor	Legacy: paper management, reviewer assignment	Yes
admin	/admin	users list, ban/unban, role change, platform stats, submission analytics	Yes (ADMIN)
ai	/api/ai	summarize PDF, ask question about paper	No
feeds	/api/feeds	RSS/Atom feeds (latest, by journal, by discipline)	No
authors	/api/authors	author profile by ID	No

4.3 Request Lifecycle & Middleware Pipeline



4.4 Authentication & Authorization

Token Strategy

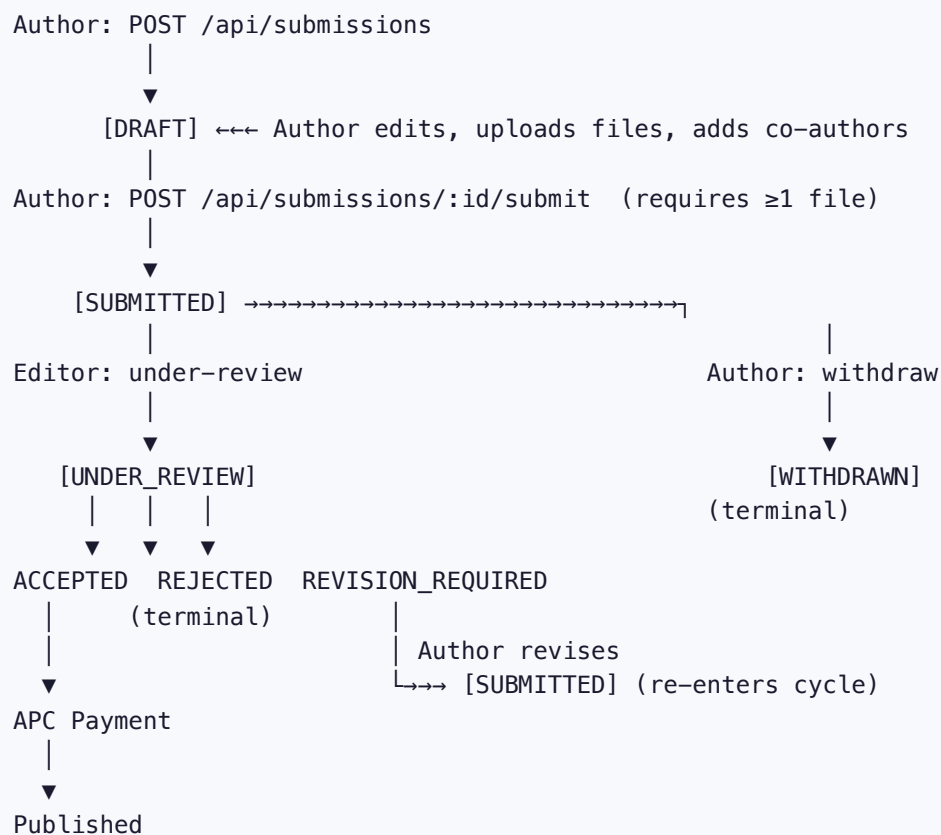
Token	Type	Lifetime	Storage	Rotation
Access Token	JWT (signed)	15 minutes	Client memory / localStorage	No (stateless)
Refresh Token	Opaque string	30 days	Database (refresh_tokens table)	Yes (rotated on use)

Role Permissions Matrix

Action	READER	AUTHOR	REVIEWER	EDITOR	ADMIN
Browse content	✓	✓	✓	✓	✓
Submit manuscript		✓			
Review papers			✓		
Editorial decisions				✓	✓
Assign reviewers				✓	✓
Publish articles				✓	✓
Manage users					✓
Platform statistics					✓
Ban/unban users					✓
Purchase articles	✓	✓	✓	✓	✓
Set alerts	✓	✓	✓	✓	✓

4.5 Workflow State Machines

Modern Submission Workflow (8 States)



Email Notifications per Transition

Every state transition automatically triggers a fire-and-forget email to the relevant party (author, reviewer, or editor). The system sends **19 different email types** across all workflows.

4.6 Caching Strategy

Cache Key Pattern	TTL	Invalidated By
homepage:main	60 seconds	Article published
top:{rankBy}:{limit}:{discipline}	2 minutes	Article published
metrics:{doi}	30 seconds	View/download increment
site-config:all	10 minutes	setSiteConfig()
search-suggestions:{q}	60 seconds	Natural expiry

4.7 Rate Limiting

Limiter	Window	Max Requests	Scope
globalLimiter	15 min	200	All routes
authLimiter	15 min	10	/api/auth/*
searchLimiter	1 min	60	/api/search/*
contactLimiter	60 min	5	/api/contact
uploadLimiter	15 min	20	File upload routes

4.8 AI Module

Provider: Groq — Model: llama-3.3-70b-versatile

Features: PDF text extraction (pdf-parse v1.1.1) → Structured summarization & Q&A

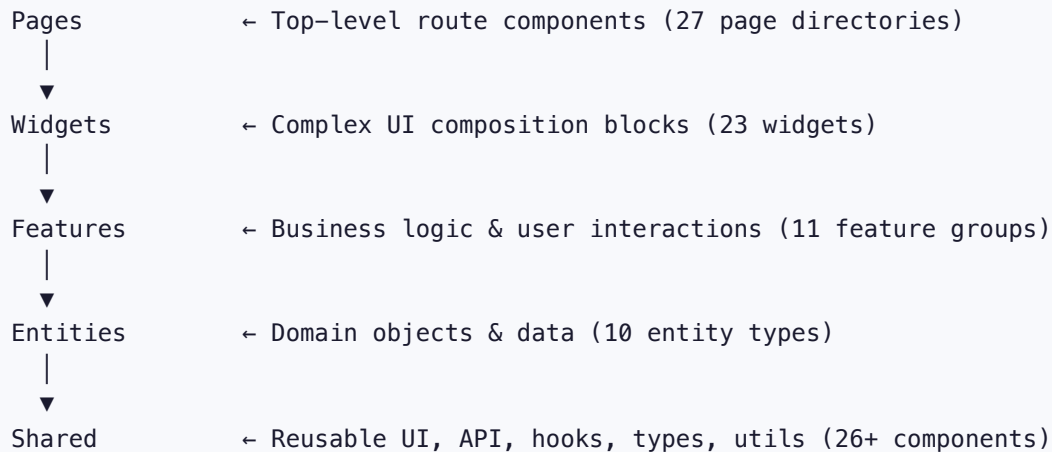
Endpoints: POST /api/ai/summarize (upload PDF → summary) & POST /api/ai/ask (upload PDF + question → answer)

The AI module extracts up to 12,000 characters from uploaded PDFs, sends the text to Groq with a structured prompt, and returns a JSON response with title, abstract, key findings, methodology, and conclusions.

5. Frontend Architecture

5.1 Feature-Sliced Design (FSD)

The frontend follows the **Feature-Sliced Design** architectural pattern, organizing code into strict layers with a one-way dependency flow:



Rule: Each layer may only import from layers below it. Pages import Widgets, Widgets import Features, etc.

5.2 Routing (50+ Routes)

Public Routes

Route	Page Component	Description
/	HomePage	Hero, featured content, metrics, news
/search	SearchResultsPage	Full-text search with faceted filters
/article/:slug	ArticlePage	Article metadata and abstract
/article/:slug/read	ArticleReadingPage	Full-text PDF/DOCX reader
/journals	JournalListPage	Browse all journals
/journal/:slug	JournalHomePage	Journal landing with nested routes
/books	BookListPage	Browse all books
/book/:isbn	BookPage	Book detail with chapters
/collections	CollectionsPage	Research collections

Route	Page Component	Description
/news	NewsPage	Publishing news and announcements
/conference/:slug	ConferencePage	Conference details
/contact	ContactPage	Contact form
/disciplines	DisciplinesPage	Browse by academic discipline
/author/:authorId	AuthorPage	Author profile and publications

Protected Routes (Requires Authentication)

Route	Required Role	Description
/account	Any authenticated	User dashboard, profile, orders
/journal/:slug/submit	Author+	Manuscript submission wizard
/checkout/:doi	Any authenticated	Article purchase checkout
/checkout-apc/:id	Author+	APC payment for publishing
/reviewer	Reviewer, Admin	Reviewer dashboard
/reviewer/review/:id	Reviewer, Admin	Submit peer review
/editor	Editor+	Editor dashboard
/editor/decision/:id	Editor+	Accept/reject/revise decision
/editor/journals	Chief-Editor, Admin	Journal management panel

5.3 State Management

Zustand Stores (Client State)

Store	Purpose	Persisted
useAuthStore	User session, tokens, institutional access	Yes (localStorage)
useSearchStore	Search query, filters, pagination, sorting	No
useUIStore	Global modal open/close state	No
useJournalAdminStore	Journal CRUD, special issues management	Yes
useSubmissionStore	Multi-step submission form state	No
useThemeStore	Dark/light mode toggle	Yes

TanStack Query (Server State)

Default stale time: 5 minutes — **Retry:** 1 attempt — **Refetch on focus:** Disabled
Entity-level query hooks: articles, journals, books, collections, conferences, users, search

5.4 API Layer & Fallback Pattern

The frontend uses a unique **fetchWithFallback** pattern for development flexibility:

```
async function fetchWithFallback<T>(  
  endpoint: string,           // e.g. "/articles"  
  fallbackJsonPath: string,  // e.g. "/mock-data/articles.json"  
  options?: RequestInit  
) : Promise<T>  
  
// 1. Tries real API call: GET /api/articles  
// 2. If fails (network error, 4xx, 5xx):  
//    Falls back to:      GET /mock-data/articles.json  
// 3. Returns typed data either way
```

This allows the frontend to function even without the backend running, using static JSON files in `/public/mock-data/`.

5.5 Component Hierarchy

Layer	Count	Examples
Shared UI	26+	Button, Input, Modal, Toast, Spinner, Skeleton, Tabs, Accordion, Pagination, Badge
Entities	10 groups	ArticleCard, JournalCard, BookCard, ChapterCard, CitationDisplay, AuthorList
Features	11 groups	SearchBar, SubmissionWizard steps, ReviewForm, DecisionForm, CitationTools, FigureViewer
Widgets	23	GlobalHeader, GlobalFooter, MegaMenu, HomeHero, TrendingResearch, MetricsPanel, SearchFilterPanel
Pages	27 dirs	HomePage, SearchResultsPage, ArticlePage, JournalHomePage, AccountDashboard, EditorDashboard

5.6 Design System

Color Palette (CSS Custom Properties)

Token	Value	Usage
<code>--lumex-accent</code>	<code>#3b5bdb</code>	Primary brand color, links, buttons
<code>--lumex-accent-dark</code>	<code>#2f4bc7</code>	Hover states
<code>--lumex-red</code>	<code>#e8181d</code>	Errors, destructive actions
<code>--lumex-oa-gold</code>	<code>#f5a500</code>	Open Access badges
<code>--lumex-text</code>	<code>#0f1523</code>	Primary text
<code>--lumex-text-muted</code>	<code>#5a6380</code>	Secondary text
<code>--lumex-bg</code>	<code>#f0f2f7</code>	Page background
<code>--lumex-card</code>	<code>#ffffff</code>	Card surfaces

Typography

Font	Usage
Plus Jakarta Sans	Primary sans-serif (headings, body)
Fraunces	Serif accent (article titles, quotes)
Source Code Pro	Monospace (code blocks, DOIs)

Dark Mode

Implemented via Tailwind's `darkMode: 'class'` strategy. The `useThemeStore` toggles a class on `document.documentElement`, and CSS variables swap accordingly.

5.7 Code Splitting Strategy

Chunk	Contents
react-vendor	react, react-dom, react-router-dom
ui-vendor	Radix UI components (dialog, tabs, accordion, etc.)
query-vendor	@tanstack/react-query
pdf-vendor	pdfjs-dist, react-pdf
math-vendor	katex, react-katex
framer-vendor	framer-motion

All page routes use `React.lazy()` with `Suspense` for automatic code splitting.

6. Database Design

6.1 Schema Overview (24 Models, 8 Enums)

Users & Authentication

Model	Key Fields	Purpose
users	id, name, email, password, role, isBanned	User accounts with role-based access
refresh_tokens	id, token, expiresAt, isRevoked, userId	Rotatable refresh tokens
password_reset_tokens	id, token, expiresAt, used, userId	One-time password reset links

Publishing Content

Model	Key Fields	Purpose
journals	slug, title, discipline, issn, impactFactor, isOpenAccess	Academic journals
journal_issues	journalId, volume, issue, year	Journal volumes/issues
articles	doi, title, abstract, keywords, pdfUrl, viewCount, citationCount	Published articles
article_authors	articleId, firstName, lastName, orcid, isCorresponding	Article author list
article_references	articleId, text, doi, order	Article bibliography
article_metrics	articleId, viewCount, downloadCount, citationCount	Article-level analytics
collections	slug, title, discipline, isActive	Curated article collections
books	isbn, title, discipline, isOpenAccess	Academic books
book_chapters	bookId, doi, title, abstract, pdfUrl	Book chapters

Submission Workflow

Model	Key Fields	Purpose
submissions	title, abstract, status, journalSlug, coverLetter	Manuscript submissions (8 statuses)

Model	Key Fields	Purpose
submission_files	submissionId, fileName, fileUrl, mimeType	Uploaded manuscripts
submission_authors	submissionId, firstName, lastName, email	Co-author information

Platform Content

Model	Purpose
news	Publishing news and announcements
careers	Job listings (department, type, closing date)
conferences	Academic conferences (status, dates, discipline)
site_config	Key-value platform configuration
homepage_content	Homepage section data (JSON)
contact_messages	Contact form submissions with ticket IDs

User Activity

Model	Purpose
alerts	User alert subscriptions (keyword, journal, discipline)
orders	Purchase records (article access, APC payments)

6.2 Key Enums

Enum	Values
Role	READER, AUTHOR, REVIEWER, EDITOR, ADMIN
SubmissionStatus	DRAFT, SUBMITTED, UNDER_REVIEW, REVISION_REQUIRED, ACCEPTED, REJECTED, WITHDRAWN
PaperStatus	DRAFT, SUBMITTED, APPROVED, REJECTED
AccessType	OPEN_ACCESS, SUBSCRIPTION, FREE
ArticleType	RESEARCH_ARTICLE, REVIEW, EDITORIAL, LETTER, CASE_STUDY, BOOK_REVIEW, CORRECTION
ConferenceStatus	UPCOMING, ONGOING, PAST, CANCELLED
OrderStatus	PENDING, COMPLETED, FAILED, REFUNDED
AssignmentStatus	PENDING, COMPLETED

7. Frontend-Backend Integration Status

7.1 Connected Endpoints Working

Feature	Frontend	Backend	Status
Auth (login, register, logout, me)	useAuthStore	/api/auth/*	Connected
Articles (list, detail, top)	articleQueries	/api/articles/*	Connected
Journals (list, detail, articles)	journalQueries	/api/journals/*	Connected
Books (list, detail, chapters)	bookQueries	/api/books/*	Connected
Collections (list, detail)	collectionQueries	/api/collections/*	Connected
Search (full-text, suggestions)	searchQueries	/api/search/*	Connected
News (list, detail)	newsQueries	/api/news/*	Connected
Homepage data	homepageQueries	/api/homepage	Connected
Site configuration	useSiteConfig	/api/site-config	Connected

7.2 Partially Connected Fallback Active

Feature	Issue
User Dashboard	Falls back to <code>user.json</code> mock data for dashboard content
Submissions	Backend exists but some query hooks use mock data
Checkout / Orders	Backend stubbed; frontend has pages but no real payment flow
Conferences	Backend endpoint exists; frontend uses mock JSON exclusively
Special Issues	Falls back to <code>special-issues.json</code> on API failure

7.3 Not Connected Backend Only

Backend Module	Endpoints	Frontend Status
AI (summarize, Q&A)	/api/ai/*	No UI page exists
Careers	/api/careers/*	No page component, no route
RSS Feeds	/api/feeds/*	No feed links in UI
Admin (full)	/admin/*	No admin dashboard page

Backend Module	Endpoints	Frontend Status
Contact (admin view)	/api/contact/admin	No admin inbox UI
Legacy paper workflow	/papers/*, /reviews/*, /editor/*	Not connected via /api prefix

Integration Summary: Approximately 50-60% of the frontend is connected to real backend APIs. The remaining pages use mock JSON data through the `fetchWithFallback` pattern. A 7-phase migration plan has been created in `Workflow/taskfile.md` with 134 tasks to achieve 100% integration.

8. Testing & Quality Assurance

8.1 Backend Tests

Test Suite	Passed	Failed	Coverage
Basic API tests (test-api.sh)	61	0	Auth, CRUD, search, pagination
Advanced API tests (test-api-advanced.sh)	125	1	Workflows, edge cases, validation
Database integration (db.test.ts)	57 tests / 15 groups	0	All Prisma models and relations
Total	243	1	99.6% pass rate

8.2 Frontend Tests

Framework	Type	Status
Vitest	Unit tests	Configured, minimal coverage (2 test files)
Playwright	E2E tests	Configured, 3 specs (home, a11y, mobile)
axe-core	Accessibility	Integrated with Playwright

8.3 Code Quality Tooling

Tool	Purpose	Enforcement
TypeScript (strict)	Type safety	Build-time error checking
ESLint	Code linting	Pre-commit hook via Husky
Prettier	Code formatting	Pre-commit hook via lint-staged
Zod	Runtime validation	Every request body & query validated

9. Deployment & Environment

9.1 Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	Neon PostgreSQL connection string
JWT_SECRET	Yes	Access token signing secret
SUPABASE_URL	Yes	Supabase project URL
SUPABASE_SERVICE_KEY	Yes	Supabase service role key (file storage)
SUPABASE_BUCKET	Yes	Storage bucket name
RESEND_API_KEY	Yes	Resend transactional email key
EMAIL_FROM	Yes	Sender email address
APP_URL	Yes	Frontend base URL for email links
GROQ_API_KEY	No	Groq AI API key (for AI features)
PORT	No	Server port (default: 5000, use 4000 on macOS)
NODE_ENV	No	development or production
CORS_ORIGINS	No	Comma-separated allowed origins

9.2 Development Commands

Command	Description
<code>npm run dev</code> (backend)	Start backend with tsx watch (hot reload)
<code>npm run dev</code> (frontend)	Start Vite dev server with HMR
<code>npx prisma generate</code>	Generate Prisma client from schema
<code>npx prisma db push</code>	Push schema to database
<code>npx prisma db seed</code>	Seed database with sample data
<code>npx prisma studio</code>	Open Prisma database GUI
<code>npm run build</code>	TypeScript compile + Vite production build
<code>npm test</code>	Run test suite

10. Roadmap & Next Steps

A detailed 7-phase integration plan exists in `Workflow/taskfile.md` (134 tasks). Summary:

Phase	Focus	Tasks	Priority
Phase 1	Foundation & Auth Wiring — remove mock fallbacks, complete auth flow	21	Critical
Phase 2	User Profile & Dashboard — replace user.json with real APIs	18	Critical
Phase 3	Submission & Peer Review — complete author/reviewer/editor workflows	23	High
Phase 4	Admin Panel — user management, content CRUD, analytics	22	High
Phase 5	Payment & Checkout — Stripe/Razorpay integration, APC flow	12	Medium
Phase 6	AI Features & Advanced — summarizer UI, careers page, RSS, contact	18	Medium
Phase 7	Polish & Production — remove all mocks, caching, security, deploy	20	Final

Key Milestones

- Milestone 1 (Phase 1-2):** All public pages serve real data; user accounts fully functional
- Milestone 2 (Phase 3):** End-to-end submission → review → editorial decision workflow
- Milestone 3 (Phase 4):** Admin can manage users, content, and platform settings
- Milestone 4 (Phase 5-6):** Payments live; AI features accessible; all backend modules exposed
- Milestone 5 (Phase 7):** Production deployment with zero mock data dependencies

Immediate Prerequisites

- Obtain and configure **Supabase Service Role Key** (needed for file uploads)
- Obtain and configure **Resend API Key** (needed for email notifications)
- Set up Vite dev server proxy to backend (`PORT=4000`)